

Reflection Essay

When I first read the assignment, I thought it would be a straightforward application of the projective geometry concepts discussed in the class. However, after actually working through the task, I realized that the gap between mathematical theory and real-world images is much larger than I initially expected.

The first part of the assignment involved estimating the distance between a bicycle and the Arch. First to locate the bicycle, I used the Ultralytics implementation of the YOLOv8n object detection model. I selected the pretrained YOLOv8n model because it is lightweight and easy to run on a personal computer. The bicycle class corresponds to class ID 1 in the COCO dataset on which YOLOv8 is trained.

My first attempt was unsuccessful. Running the detector using the default settings produced no bicycle detections. At first, I thought there was a bug in my code, after inspecting I realized that the bicycle occupies only a very small number of pixels. The detector was simply unable to recognize such a tiny object at the default resolution. After some experimentation, I increased the inference image size to 1280 pixels and reduced the confidence threshold to 0.05. This finally produced a bicycle detection with a low but usable confidence score. This was an important lesson because it showed that even state-of-the-art deep learning models can struggle with tiny objects.

After detecting the bicycle, I selected the bottom-center point of the box as the bicycle's ground contact point. This decision was important because the center of the bounding box does not necessarily lie on the ground plane.

The next challenge was identifying the vanishing point and constructing the projective line. In classroom examples, the vanishing point is usually obvious and precisely defined. In a real image, however, multiple lines appear to converge at slightly different locations. I used the edges of the road and surrounding structures to estimate the vanishing point and drew a reference line passing through the arch, bicycle, MB reference line, and vanishing point. Small changes in the vanishing point position produced noticeably different results, which demonstrated how sensitive single-view metrology can be to measurement errors.

The real-world scale was obtained using Google Maps. I measured the distance between the Main Building reference line and the Arch of Knowledge and used this as the known reference distance.

The second part of the assignment was much more enjoyable because it involved collecting my own data. I took photograph on stairs. I asked my sister to take third-person image showing the entire scene. Several objects were available in the environment, including a plastic stool, a mobile phone and railing. I eventually selected

the stool as the reference object because it was clearly visible and approximately aligned with me on the same ground plane.

Capturing suitable photographs turned out to be more difficult than expected. I took multiple pictures before obtaining images that satisfied the assumptions of projective geometry.

For detecting person in the image, I again used YOLOv8n. This time the detector performed significantly better because the person occupied a larger region of the image. The model successfully detected the person with high confidence. I also attempted to detect the stool automatically. However, unlike the person, the stool was not consistently detected. This highlighted another practical challenge: deep learning models perform well only when the object appearance closely matches the examples seen during training.

A major takeaway from this assignment was the importance of assumptions. The mathematical derivation of cross-ratio assumes a perfectly planar ground surface, accurate vanishing point estimation, and exact point localization. Real-world images violate all of these assumptions to some extent. The road may not be perfectly flat, object detectors may produce imperfect bounding boxes, and manual point selection inevitably introduces errors. As a result, distance estimates should be interpreted as approximations rather than exact measurements.

The software tools used in this assignment included Python, Ultralytics YOLOv8, OpenCV, Google Maps, and MS Paint. Python and YOLOv8 were used for object detection, OpenCV was used for image processing and annotation, Google Maps provided real-world reference distances, and MS Paint was used to mark points and draw the projective reference line. Together, these tools allowed me to bridge the gap between computer vision and projective geometry.

Overall, this assignment was one of the most interesting applications of projective geometry that I have encountered. It demonstrated how concepts such as vanishing points, perspective projection, and cross-ratio can be used to extract meaningful real-world measurements from a single image. At the same time, it exposed the practical challenges of working with real photographs, noisy detections, and imperfect assumptions. The combination of deep learning and projective geometry made this assignment both challenging and rewarding, and it significantly improved my understanding of how real-world distance estimation can be performed from a single image.